AQI Health Advisor

Software Lab-I Mini Project | III Semester CSE

1. Problem Statement

Air pollution is a major environmental problem that affects human health. Different pollutants present in the air can cause respiratory and other health problems. The Air Quality Index (AQI) helps us understand how polluted the air is.

The **AQI Health Advisor** is a Python-based project that analyses air quality data for different Indian cities. It allows users to check the AQI of a city on a particular date, identify its AQI category, and receive health advice according to the pollution level. The project also analyses pollution trends, compares cities, and studies the relationship between different pollutants and AQI.

2. Dataset Description

The project uses the `city_day.csv` dataset, which contains air quality information for Indian cities.

The dataset includes the following important columns:

- **City:** Name of the city.
- **Date:** Date on which the observation was recorded.
- **PM2.5 and PM10:** Particulate matter pollutants present in the air.
- **NO2, CO and other pollutants:** Different gases and pollutants measured in the air.
- **AQI:** Air Quality Index value.
- **AQI_Bucket:** AQI category recorded in the dataset.

The dataset is loaded and analysed using Python and Pandas. Missing AQI values are removed before analysis, and additional data-cleaning steps are performed where required.

3. Work Done by Each Team Member

Role 1: Core Logic and Menu

Role 1 develops the main logic of the program. This includes the `get_aqi_category(aqi)` function, which classifies AQI values into six categories: Good, Satisfactory, Moderate, Poor, Very Poor, and Severe.

The role also develops the `get_advice(category)` function to provide health recommendations, the `AirReport` class to generate reports, file handling to save reports, and a menu-driven program that allows users to check AQI for a city and date.

Role 2: Data Structures

Role 2 demonstrates the use of Python lists, tuples, sets, strings, and dictionaries to organise and analyse the dataset.

This includes creating lists of city, date, and AQI records; performing set operations to compare cities with high PM2.5 and PM10 levels; building nested dictionaries to store monthly AQI values; and counting repeated values.

Role 3: Data Cleaning and NumPy

Role 3 examines the dataset's structure, data types, and missing values. It demonstrates replacing abnormal symbols with missing values and filling missing PM2.5 and PM10 values with their respective means.

NumPy is used to calculate statistical measures such as mean, median, standard deviation, maximum, minimum, and the 90th percentile of AQI. This role also identifies the worst AQI day for each city.

Role 4: Pandas Analysis and Report

Role 4 performs detailed data analysis using Pandas. It calculates average AQI values for different cities, identifies the most and least polluted cities, and creates a correlation matrix to examine the relationship between pollutants and AQI.

This role also calculates Pearson correlation, creates an `AQI_Category` column, counts AQI categories, identifies the top five highest-AQI records, sorts the dataset by AQI, and calculates average pollutant levels for each city. Finally, it prepares the project report and README.

4. Key Findings

City Pollution Analysis

The average AQI for each city is calculated using the Pandas `groupby()` and `mean()` functions. Cities are sorted according to their average AQI.

- **Most polluted city:** Ahmedabad
- **Least polluted city:** Aizawl

These results help compare air quality across different cities in the dataset.

Correlation Analysis

A correlation matrix is generated for PM2.5, PM10, NO2, CO, and AQI to understand how these pollutants are related to the Air Quality Index.

The analysis produced the following result:

- **Strongest pollutant link:** PM10, with a correlation coefficient of approximately 0.8033.
- **Direction of relationship:** Positive.

This indicates that PM10 has the strongest positive correlation with AQI among the four pollutants examined. Higher PM10 values tend to be associated with higher AQI values in the dataset. Correlation indicates an association and does not, by itself, prove causation.

AQI Category Analysis

The AQI values are classified into six categories using the `get_aqi_category()` function. The `value_counts()` function is used to count the number of observations in each category.

The five records with the highest AQI values are also identified to examine the most polluted observations in the dataset.

5. Tools and Technologies Used

- **Python:** Main programming language.
- **Google Colab:** Environment used to write and execute the code.
- **Pandas:** Data loading, cleaning, grouping, sorting, and analysis.
- **NumPy:** Numerical calculations and statistical analysis.
- **SciPy:** Calculation of Pearson correlation.
- **Google Drive:** Storage and access to the dataset and notebooks.

6. How to Run the Project

1. Open Google Drive and locate the shared `AQI_Project` folder.
2. Ensure that `city_day.csv` is available inside the `data` folder.
3. Open the final notebook named `AQI_Final.ipynb` in Google Colab.
4. Run the notebook cells from top to bottom.
5. Allow Google Colab to access Google Drive when prompted.
6. Ensure that the dataset is loaded and the required data-cleaning steps are completed.
7. Use the program's menu to enter a city and date and check the corresponding AQI.
8. View the AQI category and health advice displayed by the program.
9. Check the generated `aqi_report.txt` file for saved air quality reports.

Dataset path:

`/content/drive/MyDrive/AQI_Project/data/city_day.csv`

7. Conclusion

The AQI Health Advisor demonstrates how Python programming and data analysis can be used to understand air pollution. By combining functions, classes, file handling, data structures, NumPy, and Pandas, the project provides a way to classify AQI values, generate health advice, compare cities, and analyse pollutant relationships.

The project also demonstrates how data analysis can help users understand air quality information and become more aware of potential pollution-related health risks. It uses basic Python programming and statistical techniques without machine learning.